Bài Ôn tập 1

Mã bài tập	Đề bài (Rẽ nhánh, vòng lặp, hàm, Cấu trúc dữ liệu: List, Tuple, Dictionary)
1	Yêu cầu. Cước phí Taxi Thông Minh. Viết một hàm duy nhất smart_taxi_fare(car_type, start_hour, km, wait_minutes) tính và trả về total (tổng tiền phải trả, kiểu số nguyên, đơn vị VND). Các quy tắc cần cài đặt: 1. Giá mở cửa (0–0.8 km): - Xe 4 chỗ: 11,000 VND - Xe 7 chỗ: 12,000 VND 2. Giá theo km sau 0.8 km: - Xe 4 chỗ: từ 0.8→20 km tính 13,000 VND/km; trên 20 km tính 10,500 VND/km - Xe 7 chỗ: từ 0.8→20 km tính 14,000 VND/km; trên 20 km tính 12,000 VND/km 3. Phí chờ: 5 phút đầu miễn phí; sau đó, mỗi 5 phút (làm tròn lên theo từng block 5 phút) tính 2,000 VND. 4. Phụ thu ban đêm: nếu start_hour thuộc [22,23,0,1,2,3,4,5], cộng +20% trên tiền tạm tính (mở cửa + quãng đường + chờ). 5. Điều kiện đặc biệt: - Nếu km > 30 và không phải ban đêm, áp dụng giảm 10% trên tổng tiền. - Nếu tổng tiền cuối cùng < 50,000 nhưng km > 5, đặt tổng tiền bằng 50,000 VND (mức tối thiểu).
2	Yêu cầu: Cho số nguyên x, trả về True nếu tồn tại số nguyên n ≥ 1 sao cho x = 1 + 2 + ... + n, ngược lại trả về False. Khuyến khích dùng vòng lặp đơn giản để cộng dồn tổng.
3	Yêu cầu: Cho tọa độ ba điểm A(x1,y1), B(x2,y2), C(x3,y3), trả về True nếu chúng tạo thành một tam giác, ngược lại trả về False. Quy tắc: Chúng tạo thành tam giác nếu không thẳng hàng (diện tích có hướng khác 0).
4	Yêu cầu: Cho các hệ số thực a, b, c, trả về tất cả các nghiệm thực của phương trình bậc hai axx + bx + c = 0 trong một danh sách được sắp xếp tăng dần.

	Quy tắc: - Nếu a == 0 và b != 0: trả về nghiệm duy nhất [-c / b]. - Nếu a == 0 và b == 0: trả về [] (không có nghiệm hữu hạn duy nhất nếu c == 0, hoặc vô nghiệm nếu c != 0). - Với a != 0, dùng biệt thức d = bb - 4ac: - d < 0 → [] - d == 0 → [-b / (2a)] - d > 0 → hai nghiệm [r1, r2] với r1 <= r2.
5	Yêu cầu: Cho ba số a, b, c, trả về chúng dưới dạng danh sách theo thứ tự không giảm.
6	Yêu cầu: Cho năm số a, b, c, d, e, trả về True nếu tồn tại bất kỳ ba số nào trong đó bằng nhau; ngược lại trả về False. Không dùng tuple hay dictionary.
7	Yêu cầu: Cho ba số a, b, c, trả về giá trị ở giữa khi sắp xếp ba số theo thứ tự tăng dần (giá trị lớn thứ hai). Định nghĩa này vẫn đúng khi có phần tử trùng nhau (ví dụ: 2, 2, 5 → giá trị giữa là 2).
8	Yêu cầu: Cho hai ngày sinh dưới dạng (day, month, year) của người 1 và người 2, trả về 1 nếu người 1 lớn tuổi hơn (ngày sinh sớm hơn), 2 nếu người 2 lớn tuổi hơn, và 0 nếu bằng nhau. Ví dụ: 27 2 2007 so với 28 2 2007 → trả về 1 vì ngày 27 sớm hơn nên người 1 lớn tuổi hơn.
9	Yêu cầu: Cho số nguyên n, trả về: - 5n + 1 nếu n mod 3 == 1 - 6n + 2 nếu n mod 3 == 2 - 7n + 3 nếu n mod 3 == 0
10	Yêu cầu: Cho điểm số s, trả về xếp loại chữ theo quy tắc: - A+ nếu s ≥ 9 - A nếu s ≥ 8.5 - B+ nếu s ≥ 8 - B nếu s ≥ 7 - E nếu còn lại
11	Yêu cầu: Cho một chuỗi giới tính và chiều cao tính bằng mét, trả về True nếu người đó là nam và cao hơn 1.70 m; ngược lại trả về False. Chấp nhận các cách biểu diễn nam như 'male', 'm', 'nam' (không phân biệt hoa thường).

12	Yêu cầu: Cho cân nặng (kg) và chiều cao (m) của hai người, hãy trả về người có BMI cao hơn. Trả về 1 nếu người 1 có BMI cao hơn, 2 nếu người 2 có BMI cao hơn, và 0 nếu bằng nhau. Nhắc lại: BMI = weight / (height height).
13	Viết hàm trả về tổng bình phương các số từ 1 đến n.
14	Viết hàm trả về hệ số nhị thức C(n, k).
15	Viết hàm trả về giá trị lớn nhất và nhỏ nhất của ba số a, b, c dưới dạng danh sách [max, min]. (Không dùng tuple hay dictionary)
16	Viết hàm trả về góc ∠ABC theo đơn vị độ khi biết tọa độ các điểm A(x1,y1), B(x2,y2), C(x3,y3).
17	Viết hàm trả về khoảng cách Euclid giữa hai điểm A(x1, y1) và B(x2, y2).
18	Viết hàm trả về số lượng chữ số khác 0 trong n.
19	Viết hàm trả về a xor b.
20	Viết hàm trả về (a b c d) % mod.
21	Viết hàm trả về (a - b) (c + d).
22	Viết hàm trả về diện tích hình tròn với bán kính r. Dùng pi = 3.141592653589793.
23	Viết hàm trả về a lũy thừa b.
24	Viết hàm trả về n mũ 6.
25	Yêu cầu: Viết hàm point_in_quadrilateral(quad, p) trả về True nếu điểm p nằm bên trong hoặc trên biên của tứ giác quad, và False nếu ngược lại. - Đầu vào: - quad: danh sách hoặc bộ gồm đúng 4 đỉnh [(x1,y1), (x2,y2), (x3,y3), (x4,y4)] theo thứ tự (tứ giác có thể lồi hoặc lõm, nhưng không tự cắt). - p: một điểm (x, y). - Đầu ra: giá trị boolean. - Gợi ý: Cài đặt phép kiểm tra điểm trong đa giác tổng quát (ví dụ: ray casting), đồng thời tính cả trường hợp điểm nằm trên biên là ở trong.
26	Yêu cầu: Viết hàm are_collinear(x1, y1, x2, y2, x3, y3) trả về True nếu ba điểm thẳng hàng, ngược lại trả về False. - Đầu vào: ba điểm x1, y1, x2, y2, x3, y3 biểu diễn tọa độ của ba điểm. - Đầu ra: giá trị boolean.

	- Gợi ý: Dùng phép kiểm tra diện tích (tích có hướng) và một sai số nhỏ khi so sánh số thực. Các điểm nằm trên cùng một đường thẳng được xem là thẳng hàng (kể cả khi có điểm trùng nhau).
27	Yêu cầu: Viết hàm quadratic_roots_count(a, b, c) trả về số nghiệm thực của phương trình ax^2 + bx + c = 0. - Đầu vào: các số thực a, b, c. - Đầu ra: 0, 1, hoặc 2. Nếu phương trình suy biến thành 0 = 0 (tức a == 0, b == 0, c == 0), trả về float('inf'). - Quy tắc: - Nếu a != 0, dùng biệt thức D = b^2 - 4ac. - Nếu a == 0 và b != 0, đó là phương trình bậc nhất với đúng một nghiệm thực. - Nếu a == 0 và b == 0 và c != 0, phương trình vô nghiệm.
28	Yêu cầu: Viết hàm sum_halves(s) nhận chuỗi s gồm đúng 6 chữ số và trả về tổng của số nguyên 3 chữ số đầu và số nguyên 3 chữ số cuối. - Đầu vào: chuỗi s độ dài 6, chỉ gồm các chữ số 0-9. - Đầu ra: tổng kiểu số nguyên. - Ví dụ: '100200' -> 100 + 200 = 300. - Nếu đầu vào không hợp lệ (sai độ dài hoặc chứa ký tự không phải chữ số), phát sinh ValueError.
29	Yêu cầu: Viết hàm is_one_product_of_others(a, b, c) trả về True nếu một trong ba số bằng tích của hai số còn lại; ngược lại trả về False. - Đầu vào: các số a, b, c (int hoặc float). - Đầu ra: giá trị boolean. - Cần xét các trường hợp đặc biệt như số 0 và số âm.
30	Yêu cầu: Viết hàm detect_number_type(s) trả về 'int' nếu s biểu diễn một số nguyên, 'float' nếu biểu diễn một số thực dấu chấm động, ngược lại trả về 'neither'. - Đầu vào: chuỗi s. - Đầu ra: một trong các giá trị {'int', 'float', 'neither'}. - Quy tắc: - Cho phép có dấu + hoặc - ở đầu. - Số nguyên phải chỉ gồm các chữ số và không có dấu chấm thập phân (ví dụ: '100'). - Số thực phải có dấu chấm và có chữ số ở cả hai phía của dấu chấm (ví dụ: '100.0', '-3.14'). - Các chuỗi như '.5', '5.', '1e3' được xem là 'neither' trong bài này.

31	Yêu cầu: Viết hàm is_perfect_square(n) trả về True nếu n là số chính phương, ngược lại trả về False. - Đầu vào: số nguyên n (có thể âm). - Đầu ra: giá trị boolean. - Lưu ý: Số âm không phải là số chính phương trong tập số nguyên.
32	Yêu cầu: Viết hàm is_right_triangle(a, b, c) trả về True nếu a, b, c có thể là ba cạnh của một tam giác vuông, ngược lại trả về False. - Đầu vào: các số dương a, b, c. - Đầu ra: giá trị boolean. - Lưu ý: Kiểm tra bất đẳng thức tam giác và định lý Pythagore. Dùng sai số nhỏ khi so sánh số thực.
33	Yêu cầu: Viết hàm compute(a, b, op) tính a (+|-||/) b tùy theo op. - Đầu vào: các số a, b và ký tự op thuộc {'+', '-', '', '/'}. - Đầu ra: kết quả số học. - Yêu cầu thêm: - Nếu op là '/' và b == 0, phát sinh ValueError. - Nếu op không hợp lệ, phát sinh ValueError.
34	Yêu cầu: Viết hàm max_of_two(a, b) trả về số lớn hơn. - Đầu vào: các số a, b (int hoặc float). - Đầu ra: giá trị lớn nhất của a và b.
35	Yêu cầu: Viết hàm divide_with_remainder(dividend, divisor) trả về bộ (q, r) trong đó q là thương nguyên và r là số dư sao cho dividend = divisorq + r với 0 <= |r| < |divisor| (dùng ngữ nghĩa của Python với // và %). - Đầu vào: số nguyên dividend, divisor (divisor khác 0). - Đầu ra: bộ hai số nguyên (q, r). - Ví dụ: divide_with_remainder(5, 3) -> (1, 2).
36	Yêu cầu: Viết hàm is_product_divisible_by_10(a, b) trả về True nếu a b chia hết cho 10, ngược lại trả về False. - Đầu vào: số nguyên a, b (có thể âm hoặc bằng 0). - Đầu ra: giá trị boolean. - Gợi ý: Một số chia hết cho 10 nếu chữ số tận cùng là 0 (tức là chia hết cho cả 2 và 5).

37	Viết hàm trả về tam giác vuông cân cao n bằng ký tự ''. Dòng thứ i (đánh số từ 1) có i dấu '', các dòng nối với nhau bằng ký tự xuống dòng. Ví dụ với n = 4
38	Viết hàm trả về chuỗi biểu diễn hình chữ nhật N x M bằng ký tự '', các hàng cách nhau bởi ký tự xuống dòng. Ví dụ, n=2, m=3
39	Viết hàm, với đầu vào là một chữ cái thường, trả về chuỗi gồm chữ cái đó lặp lại 10 lần.
40	Viết hàm chuyển một chữ cái in hoa thành chữ thường. Nếu đầu vào không phải đúng một ký tự, phát sinh ValueError.
41	Viết hàm trả về ký tự đứng sau ký tự x n vị trí trong bảng chữ cái. Giữ nguyên chữ hoa/chữ thường và quay vòng trong phạm vi A..Z hoặc a..z.
42	Viết hàm trả về True nếu a, b, c có thể là độ dài ba cạnh của một tam giác; ngược lại trả về False.
43	Viết hàm tính diện tích tam giác có ba cạnh a, b, c (dùng công thức Heron). Phát sinh ValueError nếu ba cạnh không thể tạo thành tam giác.
44	Viết hàm trả về tích các số nguyên từ 1 đến n (giai thừa). Với n=0, trả về 1.
45	Viết hàm trả về số lượng số nguyên dương k sao cho 1 <= k < n và k > (n - 1) / 2. (Ví dụ: n=9 -> 4).
46	Viết hàm trả về a + b + a b.
47	Viết hàm tính giá trị (a - b) c.
48	Viết hàm trả về số gấp đôi giá trị đầu vào (ví dụ: 23 -> 46).